

FINAL PROJECT PRESENTATION

Employee Management System

Enterprise-Grade HR Solution Framework

Developed by:

Shubham Sharma Paudel

Business Problem & Scope

Core Challenges

Legacy manual management led to data fragmentation, lack of centralized audit trails, and zero verification workflows for employee growth.

- Manual credential tracking
- Opaque data visibility
- Security vulnerabilities in PII

System Objectives

Design a robust, scalable HR platform ensuring atomic transactions, row-level data security, and verifiable credential management.

- Centralized Master Data Control
- Multi-tier Role Based Access
- Strict Document Verification

Engineering Stack



Backend

Spring Boot (Java)

- Spring Security & JWT
- Spring Data JPA
- MapStruct (Mapping)
- Hibernate ORM



Frontend

React JS (Vite)

- Tailwind CSS
- React Router Dom
- Axios (Interceptor based)
- React Context API



Persistence

PostgreSQL

- Relational Integrity
- ACID Compliance
- JSONB Support
- Connection Pooling

OOP & Engineering Principles

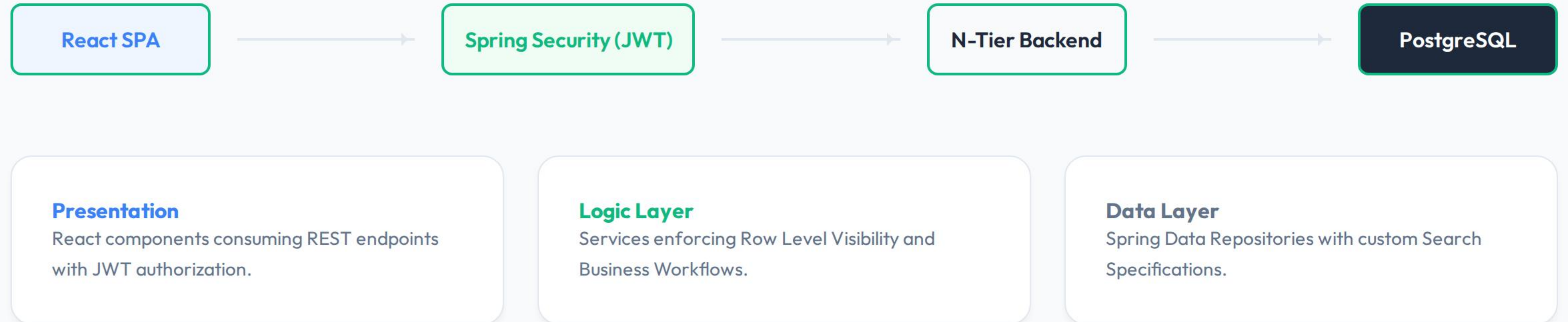
Core OOP Pillars

- **Inheritance:** BaseEntity providing common auditing fields (id, createdAt) to all Entities.
- **Abstraction:** Defining Service Interfaces decoupling implementation logic from Controllers.
- **Encapsulation:** Private entity fields protected by DTOs and selective accessors.
- **Polymorphism:** Custom response handling and dynamic security authority parsing.

SOLID Implementation

- **SRP:** Separating Business logic (Services) from API routing (Controllers).
- **OCP:** Search specifications allowing new filters without modifying core query logic.
- **DIP:** Constructor-based Dependency Injection for loose coupling.
- **DRY:** Reusable Global Exception Handler and Centralized ApiResponse wrapper.

System Architecture



Master Data Seeding

CommandLineRunner Seeding

Automated master data injection using Java-based seeders during initial startup.

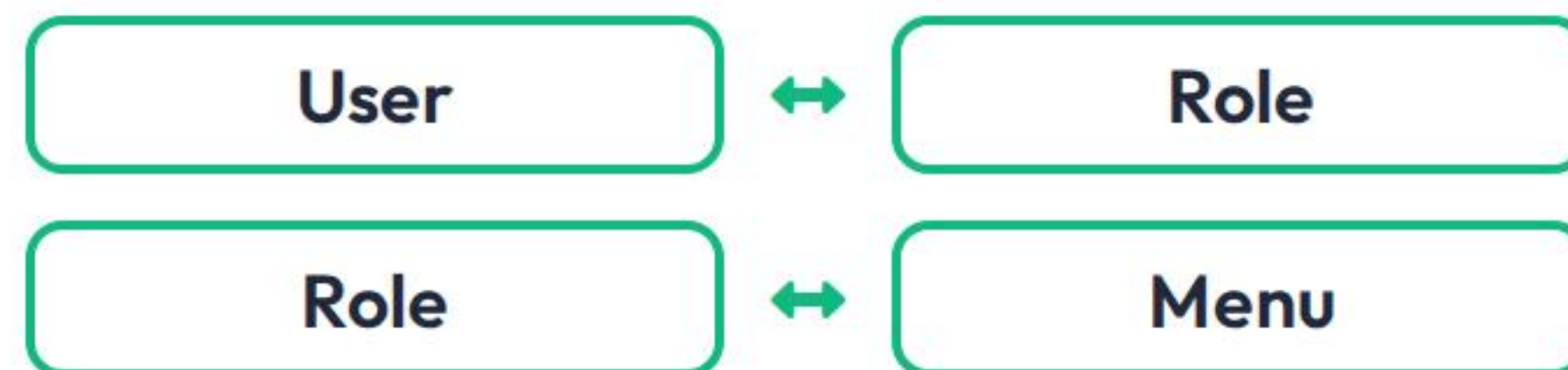
- Constraint check to prevent duplication.
- Initial HR/Admin account creation.
- Seeding: Departments, Designations, Qualification Types, and Doc Types.

```
@Component
public class DataSeeder implements CommandLineRunner {
    @Override
    public void run(String... args) {
        if(roleRepo.count() == 0) {
            // Seed Master Entities...
        }
    }
}
```


RBAC: Authorization Model

Entities & Relations

A robust decoupled model mapping users to roles and permissions via menus.



The Menu Entity

The core of our permission engine.

Attribute	Purpose
Permission Key	<code>employee:read_all</code>
Type	PAGE, LOOKUP, ACTION
Path	Dynamic Route Mapping

Permissions & Authority Mapping

STAFF ACCESS

- `employee:read_own`
- `qual:update`
- `exp:create`

DEPT HEAD ACCESS

- `employee:read_dept`
- `dept:view_reports`
- Inherits Staff authorities

ADMIN ACCESS

- `user:read` , `user:create`
- `setup:manage`
- Global Approval access

Authority Translation: Security Context holds mapped authorities directly from the `permission_key` defined in `RoleMenu` mapping.

Authentication & Password Policy

Initial State

- **Username:** Employee Email Address.
- **Temporary Pass:** 10-digit Mobile Number.
- **Boolean Flag:** `isPasswordChanged` status in User entity.

Enforcement Logic

- Interceptors check flag status before any Dashboard route.
- Mandatory redirect to `/change-password` .
- Complexity requirements: Length, Alpha-numeric, Symbols.

Transactional Onboarding

The "All or Nothing" approach for Employee Creation using **@Transactional**.

Atomic Steps:

- Validate unique Email/Phone.
- Save Employee Entity records.
- Generate User credentials (Email/Mobile).
- Map Roles & Persist UserRole.
- Commit to Database.

Failure Handling

If User creation fails (e.g., duplicate constraint), the Employee record is rolled back automatically. No orphaned employee records in the system.



Row-Level Security Enforcements

Visibility logic is **NOT** just UI hiding; it is enforced within the Service Layer using the Authenticated User Principal.



Admin

Query: `findAll()`



Dept Head

Query: `findByDept(id)`



Employee

Query: `findById(me)`

☑️ HR Approval State Engine



Rejected state: Remains pending with remarks visible for employee correction.

Search, Pagination & Performance

Dynamic Filtering

Implementing `Specification` for multi-param search.

- Name, Email, Dept, Designation
- Status & Employee Type

Spring Pageable

Preventing memory overflow with server-side pagination.

- Configurable Size: 10 to 100
- Zero-offset dynamic fetching
- $O(1)$ client side complexity



Frontend: React SPA

API Strategy

Centralized Axios client with request/response interceptors for automatic JWT handling.

Route Guards

HOC-based Protected Routes checking both Auth and Password change status.

UI Composition

Reusable forms and approval modals. Tailwind CSS for enterprise-grade responsiveness.

Dashboard: Integrated view showing Personal Details, Qualifications, and History with live Approval Badges.

Roadmap & Future Scope



Attendance

Biometric integration and Leave Management modules.



Payroll

Automated payslip generation and tax declarations.



Notifications

SMTP integration for onboarding and approval alerts.

Results: Successfully implemented a secure, scalable, and atomic HR portal meeting 100% of the project's technical requirements.

Thank You

Questions & Discussion



Secure JWT Auth



Atomic Transitions



Row-Level Visibility

Contact for Deep Dive:

Shubham Sharma Paudel